

Implementing and Testing a Probing-Based Path Verification for PolKA Source Routing Protocol

Implementação e Teste de uma Verificação de Rota Baseada em Amostragem para o Protocolo de Roteamento em Origem PolKA

Henrique Coutinho Layber

Orientadores: Roberta Lima Gomes, Magnos Martinello

Universidade Federal do Espírito Santo

20 de Março de 2025

Contexto

Em sistemas de Roteamento em Origem (SR), normalmente implementados em Redes Definidas por Software (SDNs)[1] o nó de origem define a rota que o pacote deve seguir.

Contexto

Em sistemas de Roteamento em Origem (SR), normalmente implementados em Redes Definidas por Software (SDNs)[1] o nó de origem define a rota que o pacote deve seguir.

Há uma necessidade de verificar que o pacote seguiu a rota definida pelo nó de origem, não apenas por questões de segurança, mas também para garantir que a rede esteja funcionando corretamente e configurada adequadamente.

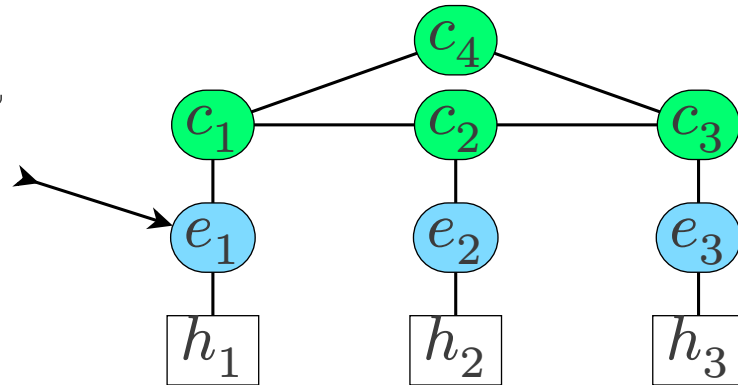
Apresentação do Problema

Assuma um pacote $h_1 \rightarrow h_3$:

1. A rota é definida pelo nó de origem (*ingress node*)
- 2.
- 3.

Define a rota

(c_1, c_4, c_3)



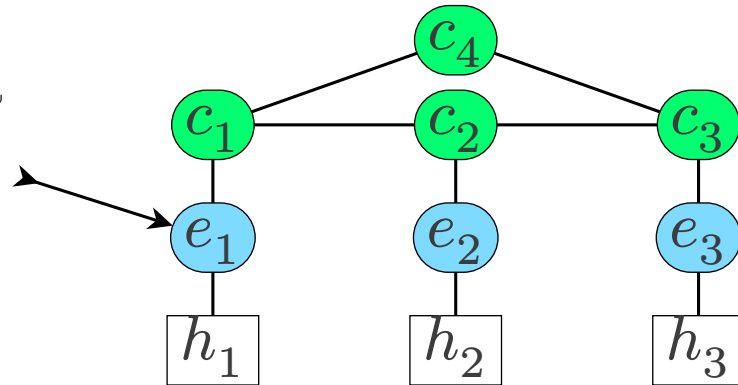
Apresentação do Problema

Assuma um pacote $h_1 \rightarrow h_3$:

1. A rota é definida pelo nó de origem (*ingress node*)
2. ???
- 3.

Define a rota

(c_1, c_4, c_3)



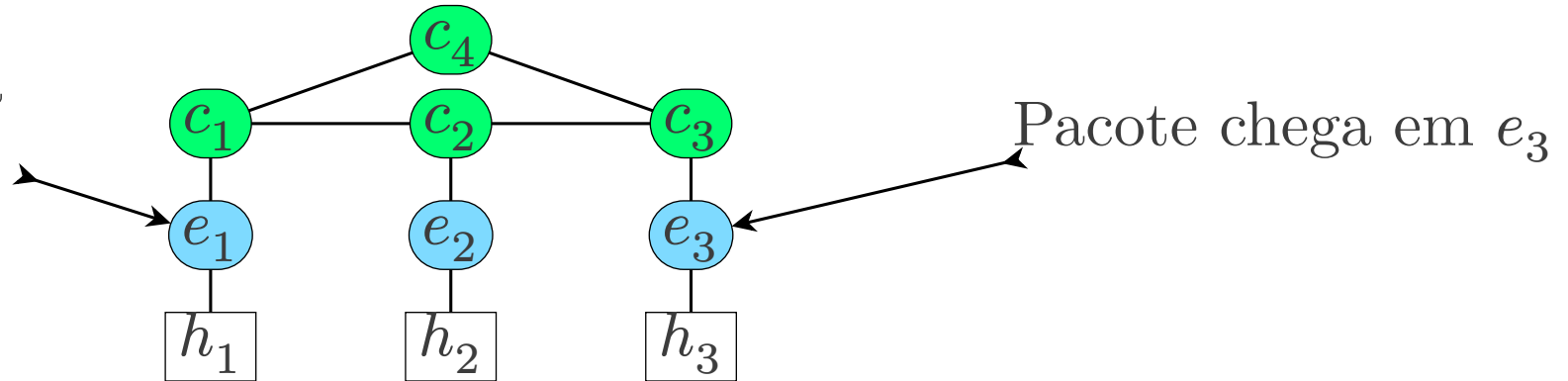
Apresentação do Problema

Assuma um pacote $h_1 \rightarrow h_3$:

1. A rota é definida pelo nó de origem (*ingress node*)
2. ???
3. O pacote chega ao nó de destino (*egress node*)

Define a rota

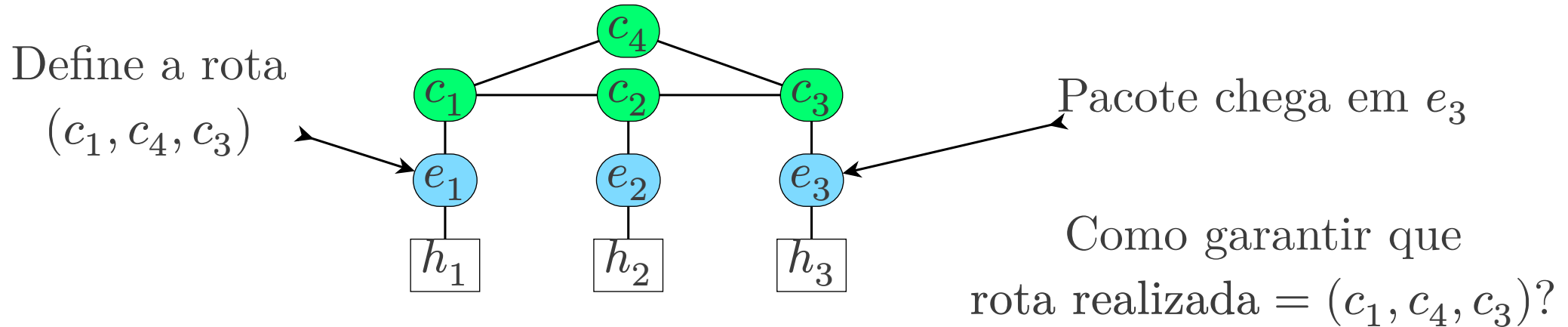
(c_1, c_4, c_3)



Apresentação do Problema

Assuma um pacote $h_1 \rightarrow h_3$:

1. A rota é definida pelo nó de origem (*ingress node*)
2. ???
3. O pacote chega ao nó de destino (*egress node*)



Formalizando o Problema

Rota definida $\underset{i \rightarrow e}{P}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Formalizando o Problema

Rota definida $\underset{i \rightarrow e}{P}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Rota realizada P_j é a sequência de nós s_r coletada $(s_{r_1}, s_{r_2}, \dots, s_{r_n})$.

Formalizando o Problema

Rota definida $\underset{i \rightarrow e}{P}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Rota realizada P_j é a sequência de nós s_r coletada $(s_{r_1}, s_{r_2}, \dots, s_{r_n})$.

Precisamos simplesmente verificar se $\underset{i \rightarrow e}{P} = P_j$

Formalizando o Problema

Rota definida $\underset{i \rightarrow e}{P}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Rota realizada P_j é a sequência de nós s_r coletada $(s_{r_1}, s_{r_2}, \dots, s_{r_n})$.

Precisamos simplesmente verificar se $\underset{i \rightarrow e}{P} = P_j$

Os hops estão corretos: $(c_1, c_4, c_3) \neq (c_1, c_2, c_3)$

Formalizando o Problema

Rota definida $P_{i \rightarrow e}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Rota realizada P_j é a sequência de nós s_r coletada $(s_{r_1}, s_{r_2}, \dots, s_{r_n})$.

Precisamos simplesmente verificar se $P_{i \rightarrow e} = P_j$

Os hops estão corretos: $(c_1, c_4, c_3) \neq (c_1, c_2, c_3)$

Não há hops a mais: $(c_1, c_4, c_3) \neq (c_1, c_4, c_4, c_3)$

Formalizando o Problema

Rota definida $P_{i \rightarrow e}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Rota realizada P_j é a sequência de nós s_r coletada $(s_{r_1}, s_{r_2}, \dots, s_{r_n})$.

Precisamos simplesmente verificar se $P_{i \rightarrow e} = P_j$

Os hops estão corretos: $(c_1, c_4, c_3) \neq (c_1, c_2, c_3)$

Não há hops a mais: $(c_1, c_4, c_3) \neq (c_1, c_4, c_4, c_3)$ Validade não interessa

Formalizando o Problema

Rota definida $P_{i \rightarrow e}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Rota realizada P_j é a sequência de nós s_r coletada $(s_{r_1}, s_{r_2}, \dots, s_{r_n})$.

Precisamos simplesmente verificar se $P_{i \rightarrow e} = P_j$

Os hops estão corretos: $(c_1, c_4, c_3) \neq (c_1, c_2, c_3)$

Não há hops a mais: $(c_1, c_4, c_3) \neq (c_1, c_4, c_4, c_3)$ Validade não interessa

Não há hops a menos: $(c_1, c_4, c_3) \neq (c_1, c_3)$

Formalizando o Problema

Rota definida $P_{i \rightarrow e}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Rota realizada P_j é a sequência de nós s_r coletada $(s_{r_1}, s_{r_2}, \dots, s_{r_n})$.

Precisamos simplesmente verificar se $P_{i \rightarrow e} = P_j$

Os hops estão corretos: $(c_1, c_4, c_3) \neq (c_1, c_2, c_3)$

Não há hops a mais: $(c_1, c_4, c_3) \neq (c_1, c_4, c_4, c_3)$ Validade não interessa

Não há hops a menos: $(c_1, c_4, c_3) \neq (c_1, c_3)$

Os hops estão na ordem correta: $(c_1, c_4, c_3) \neq (c_4, c_1, c_3)$

Formalizando o Problema

Rota definida $P_{i \rightarrow e}$ é a sequência de nós s_d : $(s_{d_1}, s_{d_2}, \dots, s_{d_n})$.

Rota realizada P_j é a sequência de nós s_r coletada $(s_{r_1}, s_{r_2}, \dots, s_{r_n})$.

Precisamos simplesmente verificar se $P_{i \rightarrow e} = P_j$

Os hops estão corretos: $(c_1, c_4, c_3) \neq (c_1, c_2, c_3)$

Não há hops a mais: $(c_1, c_4, c_3) \neq (c_1, c_4, c_4, c_3)$ Validade não interessa

Não há hops a menos: $(c_1, c_4, c_3) \neq (c_1, c_3)$

Os hops estão na ordem correta: $(c_1, c_4, c_3) \neq (c_4, c_1, c_3)$

Não é preciso conhecer os elementos da sequência!

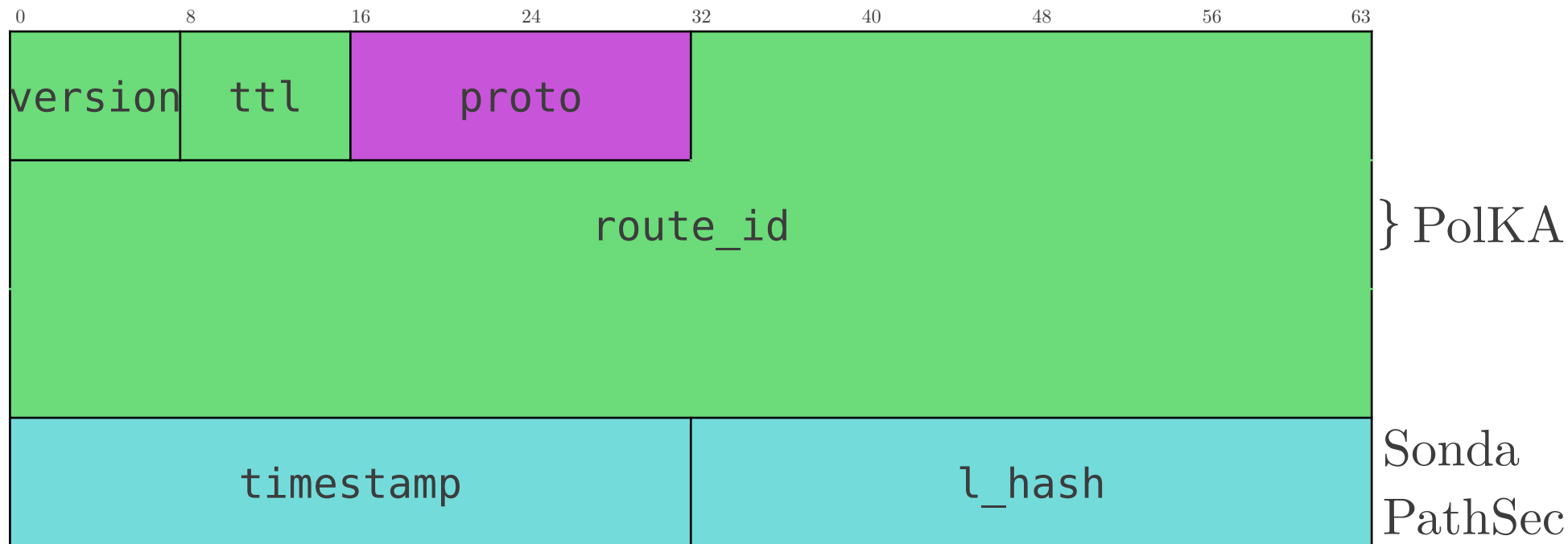
Solução proposta

PathSec[2]: Sugere uma solução baseada em amostragem como uma extensão do PolKA, alguns pacotes se tornam sondas, e recebem os campos adicionais de `timestamp` e `l_hash`. É usada uma composição de funções com um hash criptográfico, baseada no *Hash-based Message Authentication Code* (HMAC).

Solução proposta

PathSec[2]: Sugere uma solução baseada em amostragem como uma extensão do PolKA, alguns pacotes se tornam sondas, e recebem os campos adicionais de `timestamp` e `l_hash`. É usada uma composição de funções com um hash criptográfico, baseada no *Hash-based Message Authentication Code* (HMAC).

Isso define um modelo de multiassinatura. O controlador conhece os segredos dos nós (`nodeID`) e calcula a assinatura final de $P_{i \rightarrow e}$ após o *ingress edge* enviar os metadados, e o *egress edge* captura a assinatura final de P_j .

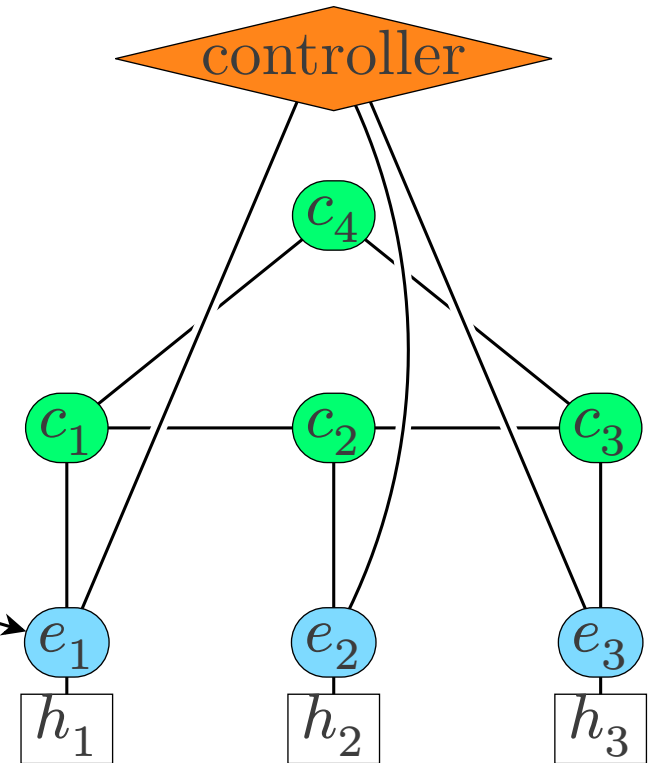


Solução proposta

$$\mathfrak{l_hash}_0 = \text{timestamp}$$

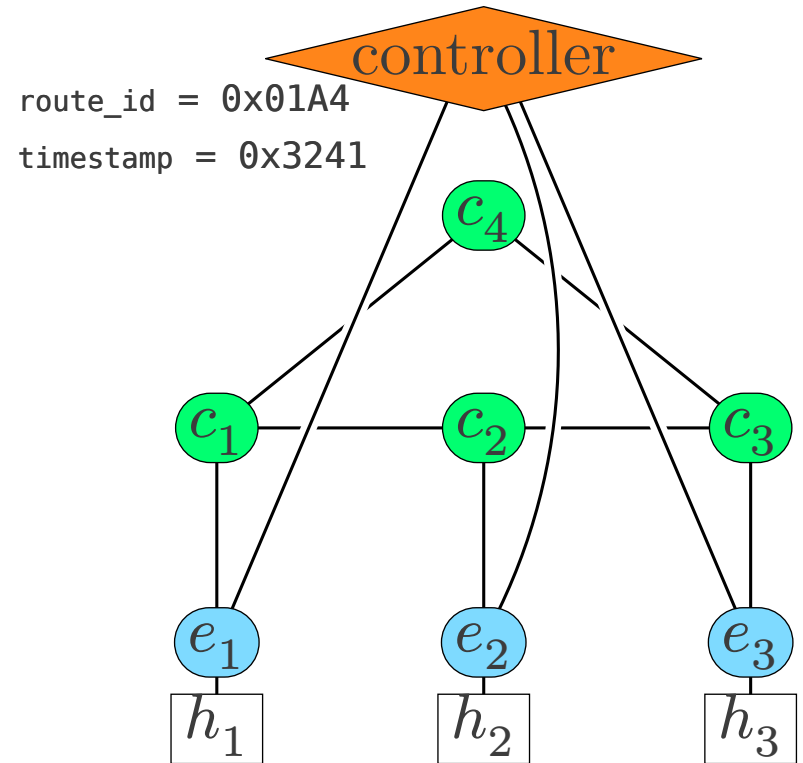
$$\mathfrak{l_hash}_n = \text{Hash}\left(\text{nodeID}_{c_n} \parallel \text{portID}_n \parallel \mathfrak{l_hash}_{n-1}\right)$$

Define a rota
 (c_1, c_4, c_3)
route_id = 0x01A4
timestamp = 0x3241



Valores ilustrativos.

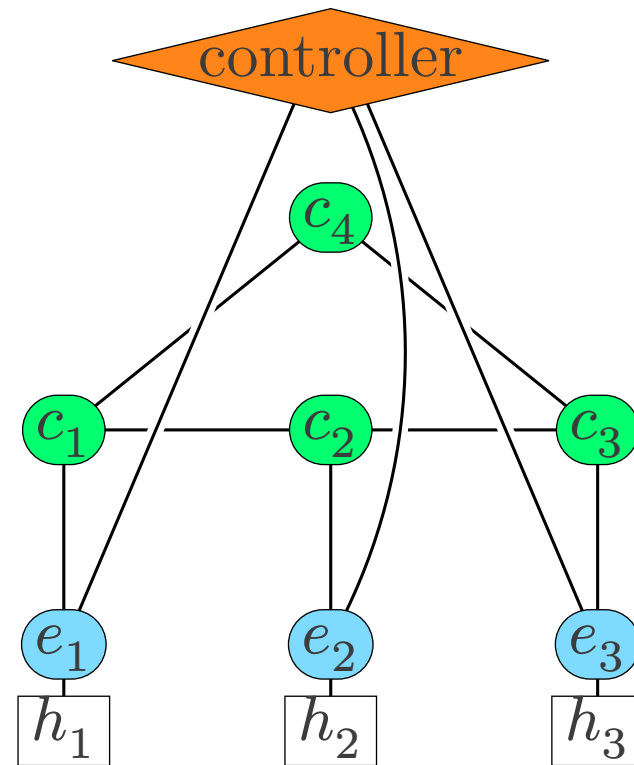
Envia metadados
ao controlador
(e também ao core)



Valores ilustrativos.

$$\mathfrak{l_hash}_n = \text{Hash}(\text{nodeID}, \dots, \mathfrak{l_hash}_{n-1})$$

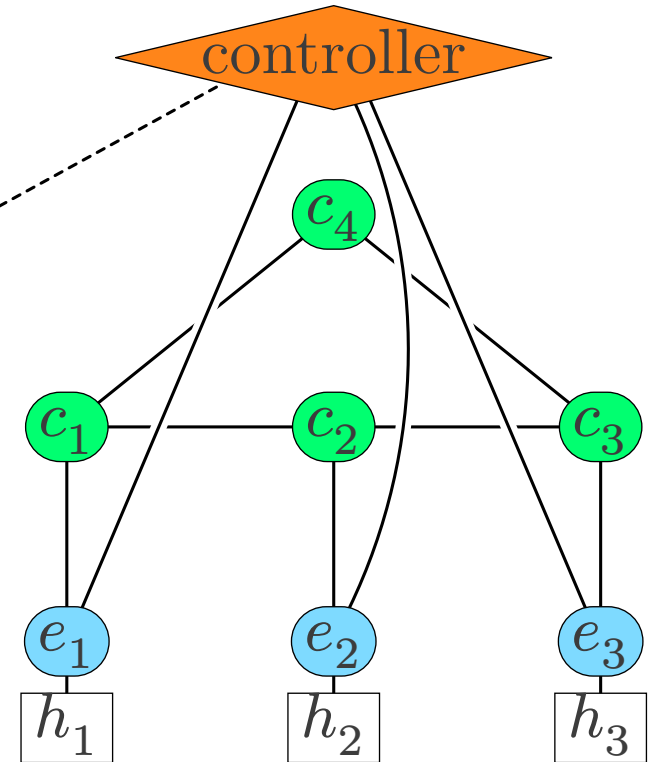
Enquanto o controlador
calcula a assinatura
de ref., o core calcula
a assinatura nativamente



Valores ilustrativos.

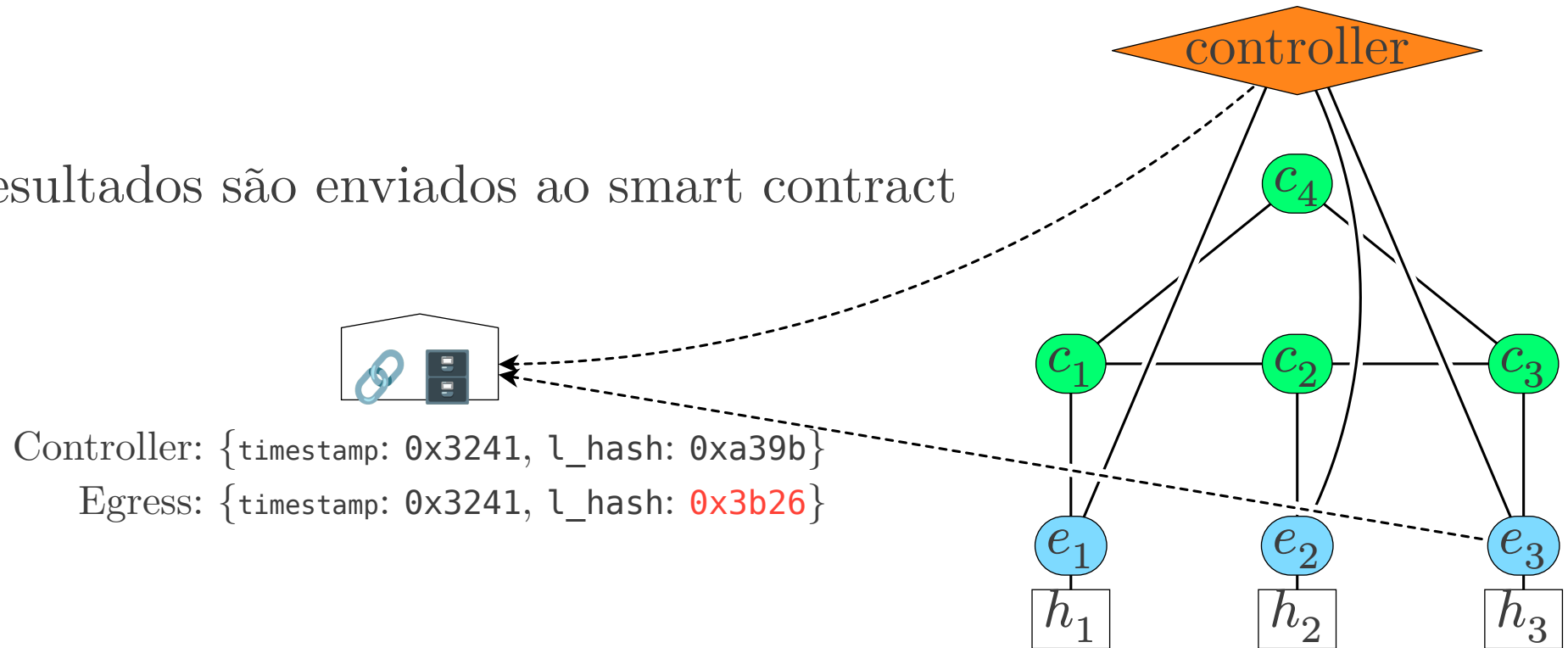
$l_hash_n = \text{Hash}(\text{nodeID}, \text{portID}, l_hash_{n-1})$
timestamp = 0x3241 route_id = 0x01A4

$c_1 : H(0x3241 \| \text{node_id}_{c_1} \dots) = 0xc206$
 $c_4 : H(0xc206 \| \text{node_id}_{c_4} \dots) = 0xe71f$
 $c_3 : H(0xe71f \| \text{node_id}_{c_3} \dots) = 0xa39b$



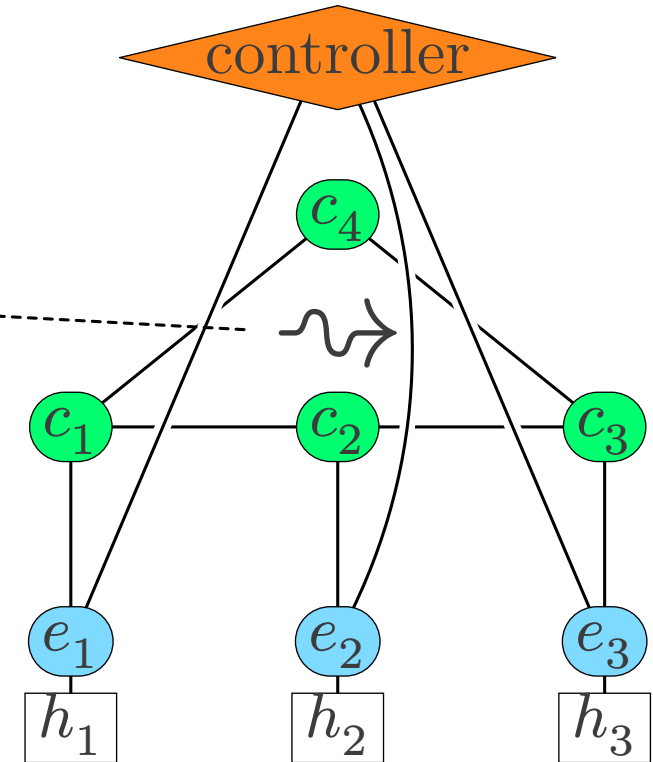
Valores ilustrativos.

Os resultados são enviados ao smart contract



Valores ilustrativos.

$$\begin{aligned} \text{l_hash}_n &= \text{Siphash}(\text{nodeID} \parallel \text{portID} \parallel \text{timestamp} \parallel 0000000, \text{l_hash}_{n-1}) \\ \text{timestamp} &= 0x3241 \quad \text{route_id} = 0x01A4 \end{aligned}$$

$$\begin{aligned} c_1 &: H(0x3241 \parallel \text{node_id}_{c_1} \dots) = 0xc206 \\ c_2 &: H(0xc206 \parallel \text{node_id}_{c_2} \dots) = 0xb38f \\ c_3 &: H(0xb38f \parallel \text{node_id}_{c_3} \dots) = \mathbf{0x3b26} \end{aligned}$$


Valores ilustrativos.

Implementando

Foi extendido de um projeto de PolKA em P4 já existente[3]. O Mininet-wifi[4] foi usado para simular a rede. Scapy[5] foi usado para fazer *sniffing* dos pacotes trafegados.

Foi utilizado um hash criptográfico *Siphash-c-d*, mais especificamente SipHash-2-4-32[6], ou seja, 2 *c-rounds* e 4 *d-rounds*, com uma chave de 64bits e entrada de 32bits.

Implementando

Foi extendido de um projeto de PolKA em P4 já existente[3]. O Mininet-wifi[4] foi usado para simular a rede. Scapy[5] foi usado para fazer *sniffing* dos pacotes trafegados.

Foi utilizado um hash criptográfico *Siphash-c-d*, mais especificamente SipHash-2-4-32[6], ou seja, 2 *c-rounds* e 4 *d-rounds*, com uma chave de 64bits e entrada de 32bits.

$$\text{l_hash}_n = \text{SipHash-2-4-32} \left(\overbrace{\text{nodeID}_{c_n}}^{16b} \parallel \overbrace{\text{portID}_n}^{9b} \parallel \overbrace{\text{timestamp}}^{32b} \parallel \overbrace{0000000}^{7b}, \overbrace{\text{l_hash}_{n-1}}^{32b} \right)$$

SipHash, c-rounds e d-rounds

SipHash é um algoritmo baseado em *Add-Rotate-Xor* (ARX) (BLAKE, ChaCHa20, etc), e uma sequência específica de operações ARX é chamado de *SipRound*.

SipHash, c-rounds e d-rounds

SipHash é um algoritmo baseado em *Add-Rotate-Xor* (ARX) (BLAKE, ChaCHa20, etc), e uma sequência específica de operações ARX é chamado de *SipRound*.

c é o número iterações *SipRounds* entre cada ingestão de um novo bloco.

SipHash, c-rounds e d-rounds

SipHash é um algoritmo baseado em *Add-Rotate-Xor* (ARX) (BLAKE, ChaCha20, etc), e uma sequência específica de operações ARX é chamado de *SipRound*.

c é o número iterações *SipRounds* entre cada ingestão de um novo bloco.

d é o numero iterações *SipRounds* após ingerir todos os blocos, e antes de produzir a saída.

SipHash, c-rounds e d-rounds

SipHash é um algoritmo baseado em *Add-Rotate-Xor* (ARX) (BLAKE, ChaCha20, etc), e uma sequência específica de operações ARX é chamado de *SipRound*.

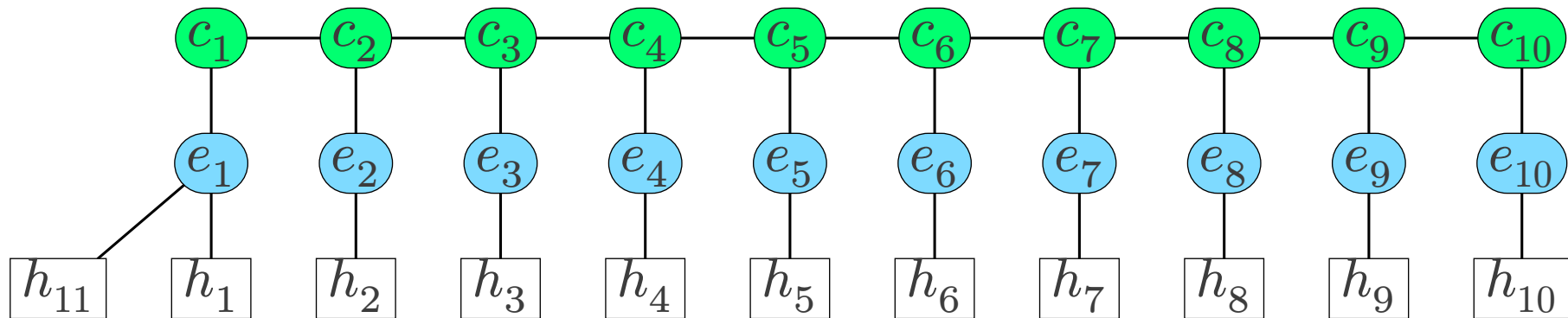
c é o número iterações *SipRounds* entre cada ingestão de um novo bloco.

d é o numero iterações *SipRounds* após ingerir todos os blocos, e antes de produzir a saída.

SipHash garante a máxima segurança MAC para $c \geq 2; d \geq 4$ [6].

Testes

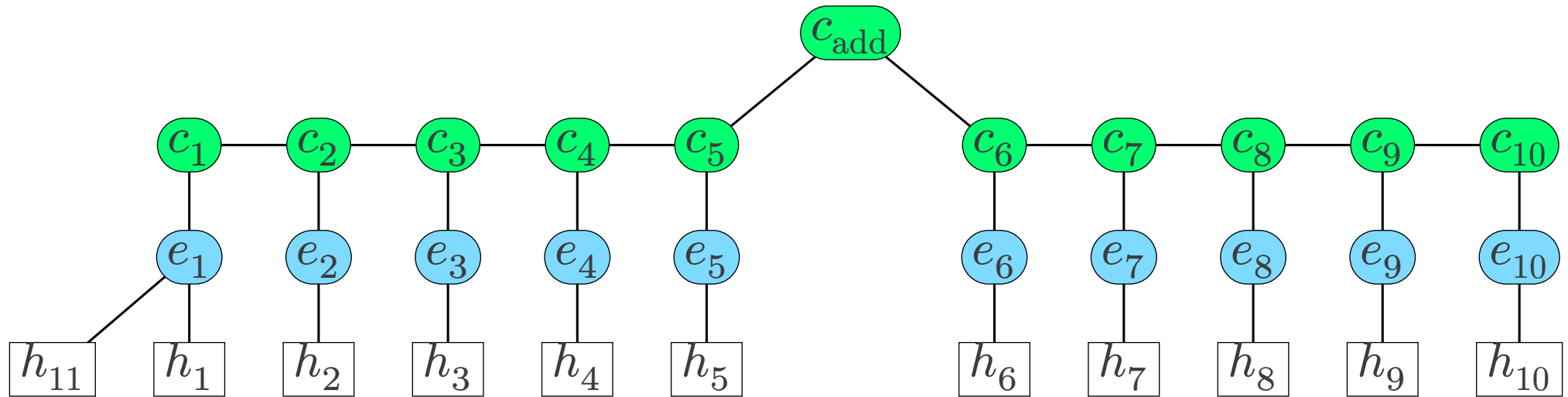
Partindo de uma topologia base



Foram testadas as seguintes condições:

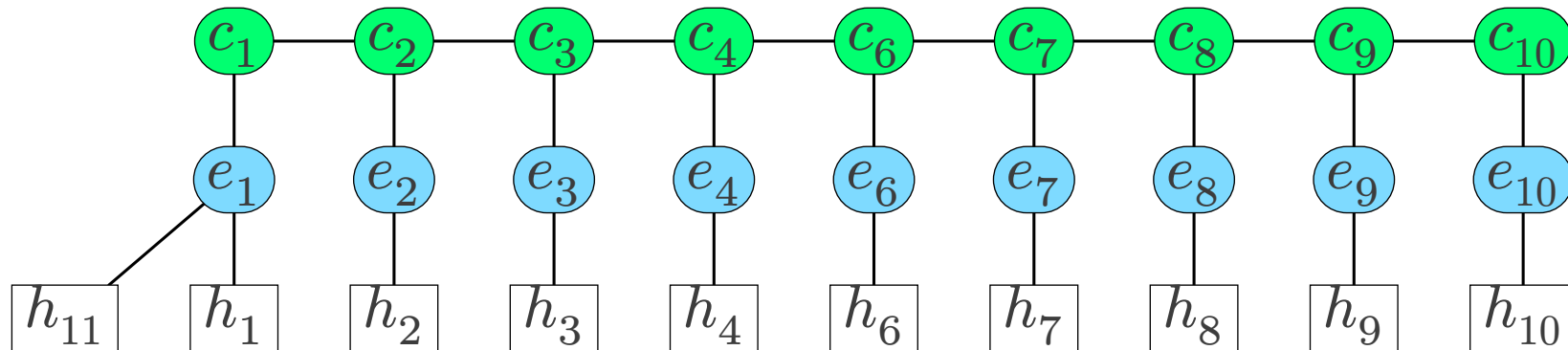
Testes

Adição



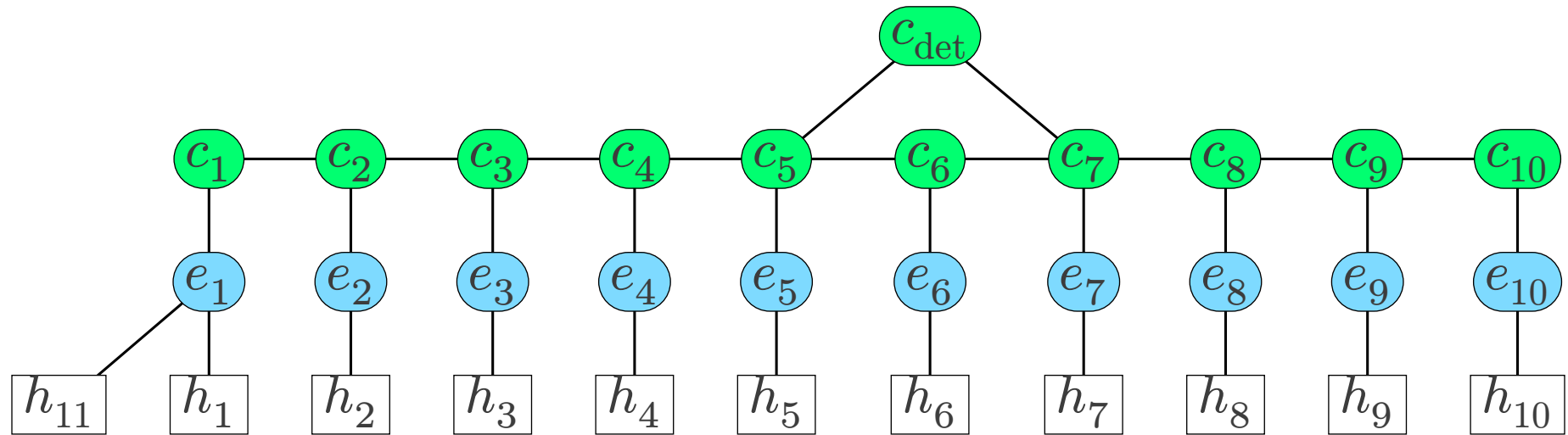
Testes

Salto



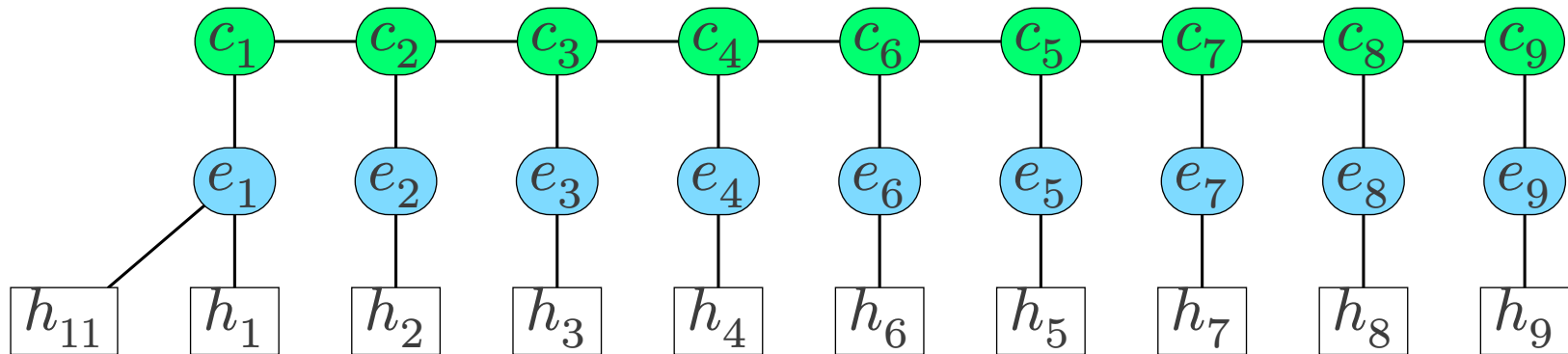
Testes

Desvio



Testes

Fora de ordem



Resultados

Para um pacote $h_1 \rightarrow h_{10}$ foram observados os seguintes resultados de `l_hash` intermediários:

(route_id = 0x707b3a1d61d1d0d8b9fc91e442d0360dfc8bba4)

Referência		Adição		Salto		Desvio		Fora de ordem	
e_1	0x61e8d6e7	e_1	0x61e8d6e7	e_1	0x61e8d6e7	e_1	0x61e8d6e7	e_1	0x61e8d6e7
s_1	0xd25dc935	s_1	0xd25dc935	s_1	0xd25dc935	s_1	0xd25dc935	s_1	0xd25dc935
s_2	0x245b7ac5	s_2	0x245b7ac5	s_2	0x245b7ac5	s_2	0x245b7ac5	s_2	0x245b7ac5
s_3	0xa3b38b83	s_3	0xa3b38b83	s_3	0xa3b38b83	s_3	0xa3b38b83	s_3	0xa3b38b83
s_4	0x26aee736	s_4	0x26aee736	s_4	0x26aee736	s_4	0x26aee736	s_4	0x26aee736
s_5	0xf9b47914	s_5	0xf9b47914			s_5	0xf9b47914	s_6	0x4b5a6c5a
		s_{add}	0x18c6d8d1						
s_6	0x18c6d8d1	s_6	0xb69b99ec	s_6	0x4b5a6c5a	s_{det}	0x250822a2	s_5	0xde3862a0
s_7	0xb69b99ec	s_7	0xfe6117f8	s_7	0x002346d3	s_7	0x40298bb9	s_7	0x648556ec
s_8	0xfe6117f8	s_8	0xc8d9fbde	s_8	0x7ec711aa	s_8	0xe13dcc9b	s_8	0x144e1d1b
s_9	0xc8d9fbde	s_9	0xa6293a25	s_9	0x5ee32b7b	s_9	0x1bf62c19	s_9	0x9e818f34
s_{10}	0xa6293a25	s_{10}	0xf4bcdcf07	s_{10}	0xc973a219	s_{10}	0xdd3a6675	s_{10}	0x6f694bc

Perguntas?

Bibliography

- [1] S. A. Jyothi, M. Dong, and P. B. Godfrey, “Towards a flexible data center fabric with source routing,” in *Proceedings of the 1st ACM SIGCOMM Symposium on Software Defined Networking Research*, in SOSR '15. Santa Clara, California: Association for Computing Machinery, 2015. doi: 10.1145/2774993.2775005.
- [2] M. Martinello *et al.*, “PathSec: Path-Aware Secure Routing with Native Path Verification and Auditability,” in *2024 IEEE Conference on Network Function Virtualization and Software*

Defined Networks (NFV-SDN), 2024, pp. 1–7. doi: 10.1109/NFV-SDN61811.2024.10807493.

- [3] C. Dominicini *et al.*, “Deploying PolKA Source Routing in P4 Switches : (Invited Paper),” in *2021 International Conference on Optical Network Design and Modeling (ONDM)*, 2021, pp. 1–3. doi: 10.23919/ONDM51796.2021.9492363.
- [4] R. R. Fontes, S. Afzal, S. H. B. Brito, M. A. S. Santos, and C. E. Rothenberg, “Mininet-WiFi: Emulating software-defined wireless networks,” in *2015 11th International Conference on Network and*

Service Management (CNSM), 2015, pp. 384–389. doi: 10.1109/CNSM.2015.7367387.

- [5] B. P. secdev, “Scapy.” [Online]. Available: <https://scapy.net/>
- [6] J.-P. Aumasson and D. J. Bernstein, “SipHash: a fast short-input PRF.” [Online]. Available: <https://eprint.iacr.org/2012/351>